

Week 3: SQL & Data Querying

Query Documentation & Analytical Insights

AnalystLab Africa Data Analytics Internship Program — Batch B

1. Overview

This report documents the SQL queries written and executed for Week 3 of the AnalystLab Africa Data Analytics Internship Program. The objective was to develop SQL querying skills across two relational datasets, covering core querying, joins, subqueries, window functions, business problem-solving, and query optimization.

Two datasets were used, as required:

Chinook Database — a digital music store dataset with tables for artists, albums, tracks, customers, and invoices.

Sales Dataset (Kaggle Sample Sales Data) — a flat sales transaction dataset (sales_data_sample.csv) covering orders, products, and customers.

Both datasets were imported into a single PostgreSQL database hosted on Supabase. The Chinook schema was created and populated using the official PostgreSQL script from the Chinook GitHub repository (lerocha/chinook-database). The sales dataset was loaded by first creating a table with explicitly defined, case-matched column names, then importing the CSV data into that existing table — this approach was adopted after the initial “create table directly from CSV” import repeatedly failed due to column name case-sensitivity mismatches between the uppercase CSV headers and the dashboard’s default lowercase column generation.

2. Database Setup

Both datasets live in the same PostgreSQL database (Supabase project: week-3-sql-analytics). Schema relationships were reviewed as follows:

Chinook: customer → invoice → invoice_line → track → album → artist, linked via foreign keys such as customer_id, invoice_id, track_id, album_id, and artist_id.

sales_data: a single denormalized table containing order, product, and customer information together (ORDERNUMBER, PRODUCTLINE, CUSTOMERNAME, SALES, ORDERDATE, and related columns).

A known data-quality consideration: the ORDERDATE column in sales_data was imported as plain text rather than a native date type, since its source format (e.g. 4/22/2005 0:00) was not reliably auto-detected. This was handled at query time using TO_DATE() rather than altering the import, which avoided import failures while still allowing accurate date-based analysis.

3. Core SQL Queries

3.1 SELECT, WHERE, ORDER BY

Business question: Which USA orders generated more than \$5,000 in sales?

```
SELECT "CUSTOMERNAME", "COUNTRY", "SALES", "ORDERDATE" FROM sales_data
WHERE "COUNTRY" = 'USA' AND "SALES" > 5000 ORDER BY "SALES" DESC;
```

Result: The Sharp Gifts Warehouse placed the highest single order value in the dataset at \$14,082.80, with several repeat large-order customers such as Online Diecast Creations Co. and

Mini Gifts Distributors Ltd. appearing frequently among the top results.

3.2 GROUP BY, HAVING, Aggregate Functions

Business question: Which customers have spent more than \$50,000 in total, and how do order frequency and average order size vary by customer?

```
SELECT "CUSTOMERNAME", SUM("SALES") AS total_sales FROM sales_data GROUP BY "CUSTOMERNAME" HAVING SUM("SALES") > 50000 ORDER BY total_sales DESC;
```

```
SELECT "CUSTOMERNAME", COUNT(*) AS num_orders, AVG("SALES") AS avg_order_value FROM sales_data GROUP BY "CUSTOMERNAME" ORDER BY num_orders DESC;
```

Result: Euro Shopping Channel is the highest-spending customer overall at \$912,294.11 — nearly 40% ahead of the second-place customer, Mini Gifts Distributors Ltd. at \$654,858.06. Revenue here is heavily concentrated: a small number of accounts (about 80 of several hundred) account for the bulk of total sales.

Notably, Euro Shopping Channel is also the most frequent buyer, placing 259 orders — far more than any other customer — yet its average order value (\$3,522.37) is lower than many smaller-volume customers. Super Scale Inc., for example, placed only 17 orders but averages \$4,674.83 per order, the highest average in the dataset. This shows two distinct paths to high revenue: Euro Shopping Channel's total is driven by order frequency, while customers like Super Scale Inc. rely on fewer, larger transactions.

4. Advanced SQL Concepts

4.1 Joins (INNER, LEFT, RIGHT)

Business question: How much has each Chinook customer spent in total, and does this change depending on join type?

```
SELECT c.first_name, c.last_name, SUM(i.total) AS total_spent FROM customer c INNER JOIN invoice i ON c.customer_id = i.customer_id GROUP BY c.customer_id, c.first_name, c.last_name ORDER BY total_spent DESC;
```

```
SELECT c.first_name, c.last_name, COALESCE(SUM(i.total), 0) AS total_spent FROM customer c LEFT JOIN invoice i ON c.customer_id = i.customer_id GROUP BY c.customer_id, c.first_name, c.last_name ORDER BY total_spent ASC;
```

```
SELECT c.first_name, c.last_name, i.invoice_id, i.total FROM customer c RIGHT JOIN invoice i ON c.customer_id = i.customer_id ORDER BY i.invoice_id;
```

Result: All three join types produced identical results across customer and invoice. This confirms clean referential integrity in the Chinook dataset — every customer has placed at least one order, and every invoice is linked to a valid customer record, so there are no orphaned rows on either side.

Top Chinook customer by spend: Helena Holý at \$49.62, closely followed by Richard Cunningham (\$47.62) and Luis Rojas (\$46.62). Spend across Chinook customers is fairly evenly distributed (roughly \$37–\$50 per customer), in clear contrast to the sales_data dataset where spend is highly concentrated among a handful of large accounts.

4.2 Subqueries

Business question: Which customers spend above the average customer spend?

```
SELECT c.first_name, c.last_name, SUM(i.total) AS total_spent FROM customer c JOIN invoice i
ON c.customer_id = i.customer_id GROUP BY c.customer_id, c.first_name, c.last_name HAVING
SUM(i.total) > ( SELECT AVG(customer_total) FROM ( SELECT SUM(i2.total) AS customer_total
FROM invoice i2 GROUP BY i2.customer_id ) AS sub ) ORDER BY total_spent DESC;
```

Result: 22 of 59 Chinook customers (about 37%) spend above the database-wide average. The subquery first computes each customer's total spend in an inner derived table, then averages those totals in a second pass, since aggregate functions cannot be nested directly (e.g. `AVG(SUM(...))` is not valid SQL within a single aggregation step).

4.3 Window Functions (ROW_NUMBER, RANK, PARTITION BY)

Business question: How should customers be ranked by spend, and how does length compare within each music genre?

```
SELECT c.first_name, c.last_name, SUM(i.total) AS total_spent, ROW_NUMBER() OVER
(ORDER BY SUM(i.total) DESC) AS spend_rank FROM customer c JOIN invoice i ON
c.customer_id = i.customer_id GROUP BY c.customer_id, c.first_name, c.last_name ORDER BY
spend_rank;
```

```
SELECT c.first_name, c.last_name, SUM(i.total) AS total_spent, RANK() OVER (ORDER BY
SUM(i.total) DESC) AS spend_rank FROM customer c JOIN invoice i ON c.customer_id =
i.customer_id GROUP BY c.customer_id, c.first_name, c.last_name ORDER BY spend_rank;
```

```
SELECT g.name AS genre, t.name AS track_name, t.milliseconds, RANK() OVER (PARTITION BY
g.genre_id ORDER BY t.milliseconds DESC) AS length_rank_in_genre FROM track t JOIN genre g
ON t.genre_id = g.genre_id ORDER BY g.name, length_rank_in_genre;
```

Result: `ROW_NUMBER` assigned a unique sequential rank to every customer, even when totals were tied (e.g. Ladislav Kovács and Hugh O'Reilly, both at \$45.62, received consecutive but distinct ranks 4 and 5). `RANK`, applied to the same data, gave both of them rank 4 and then skipped directly to rank 6 for the next customer (Julia Barnett, \$43.62) — directly illustrating how `RANK` accounts for ties by leaving a gap, while `ROW_NUMBER` never does. The largest tie observed was a 29-way tie at \$37.62, where `RANK` assigned all 29 customers rank 29 and then jumped straight to rank 59 for the next distinct value.

`PARTITION BY` was used to rank track length within each genre independently. The ranking correctly reset to 1 at every genre boundary — for example, “Reach Down” (672,773 ms) ranks 1st within Alternative, while a completely different track starts a fresh rank 1 within Alternative & Punk, and the same reset repeats across all genres through to World. Genuine ties within a genre were handled the same way `RANK` handles global ties (e.g. two Alternative & Punk tracks tied at 299,389 ms both received rank 41, with the next track skipping to rank 43), confirming partitioned ranking behaves consistently with the simple `RANK` example above. Rock is by far the largest genre by track count, with 1,297 tracks ranked, dwarfing every other genre in the catalog.

5. Business Problem Solving

5.1 Top-Performing Products / Customers

```
SELECT ar.name AS artist_name, SUM(il.unit_price * il.quantity) AS total_revenue FROM
invoice_line il JOIN track t ON il.track_id = t.track_id JOIN album al ON t.album_id = al.album_id
JOIN artist ar ON al.artist_id = ar.artist_id GROUP BY ar.name ORDER BY total_revenue DESC
LIMIT 10;
```

```
SELECT "PRODUCTLINE", SUM("SALES") AS total_revenue FROM sales_data GROUP BY
"PRODUCTLINE" ORDER BY total_revenue DESC;
```

Result: In Chinook, revenue is led by a small group of top-performing artists, identified by joining `invoice_line` through `track` and `album` up to `artist` and summing line-item revenue. Iron Maiden tops the list at \$138.60, followed by U2 (\$105.93) and Metallica (\$90.09), with Led Zeppelin close behind at \$86.13. Notably, two TV shows — *Lost* (\$81.59) and *The Office* (\$49.75) — also appear in the top 10, a reminder that Chinook's artist/album/track structure stores all media types (not just music) under the same schema. In `sales_data`, *Classic Cars* is by far the strongest-performing product line at \$3,919,615.66 in total revenue — more than double *Vintage Cars*, the second-place category at \$1,903,150.84 — while *Trains* is the weakest performer at just \$226,243.47, less than a fifth of the top category.

5.2 Revenue Trends Over Time

```
SELECT DATE_TRUNC('month', invoice_date) AS month, SUM(total) AS monthly_revenue FROM
invoice GROUP BY DATE_TRUNC('month', invoice_date) ORDER BY month;
```

```
SELECT DATE_TRUNC('month', TO_DATE("ORDERDATE", 'MM/DD/YYYY')) AS month,
SUM("SALES") AS monthly_revenue FROM sales_data GROUP BY DATE_TRUNC('month',
TO_DATE("ORDERDATE", 'MM/DD/YYYY')) ORDER BY month;
```

Result: Both queries use `DATE_TRUNC` to roll up transactions into monthly buckets. For `sales_data`, the text-based `ORDERDATE` column required conversion with `TO_DATE()` before truncation, directly applying the format ('MM/DD/YYYY') observed in the raw CSV. This enabled a like-for-like monthly revenue comparison across both datasets despite their different native date handling.

Chinook's monthly revenue is remarkably flat across the full five-year span of invoice data (January 2021 through December 2025): the overwhelming majority of months land on exactly \$37.62, with only a handful of minor deviations (e.g. \$52.62 in January 2022, \$51.62 in April 2023, \$49.62 in November 2025). There is no discernible growth trend or seasonal pattern. This strongly suggests Chinook's invoice dates were generated synthetically to spread evenly across the time window, rather than reflecting genuine purchasing seasonality — a useful reminder that Chinook is sample/demo data, not real transactional history, and shouldn't be used to draw real-world seasonal conclusions.

`sales_data` tells a very different story: a clear, repeating seasonal spike appears every November (e.g. \$1,029,837.66 in November 2003 and \$1,089,048.01 in November 2004), with October also consistently elevated each year (\$568,290.97 in October 2003, \$552,924.25 in October 2004) before revenue drops back down in December. This is a classic pre-holiday retail pattern — fitting, since the dataset covers toy and collectible sales. There is also a general upward trend year over year: 2004's baseline months (outside the November spike) run noticeably higher than 2003's equivalent months, suggesting underlying business growth on top of the seasonal effect.

5.3 Customer Purchasing Behavior

```
SELECT c.first_name, c.last_name, MIN(i.invoice_date) AS first_purchase, MAX(i.invoice_date) AS
most_recent_purchase, COUNT(i.invoice_id) AS total_purchases FROM customer c JOIN invoice i
ON c.customer_id = i.customer_id GROUP BY c.customer_id, c.first_name, c.last_name ORDER
BY total_purchases DESC;
```

Result: This query surfaces each customer's first and most recent purchase dates alongside total purchase count, distinguishing one-time buyers from repeat customers and showing how purchase activity is spread across the dataset's time range.

The result is striking: 58 of the 59 Chinook customers have made exactly 7 purchases each, spread across first-purchase dates between January 2021 and December 2022, with most recent purchases stretching out to late 2025. Only one customer, Puja Srivastava, deviates slightly with 6 purchases. This near-uniform purchase count, combined with the earlier finding of flat monthly

revenue, reinforces the conclusion that Chinook's invoice history was synthetically generated with a fixed, evenly distributed pattern per customer rather than organic, naturally varied purchasing behavior — useful context for interpreting any “customer behavior” analysis run against this dataset.

6. Query Optimization

To evaluate indexing as a performance technique, the artist-revenue join query from Section 5.1 was profiled using EXPLAIN ANALYZE before and after adding indexes on the join columns.

```
EXPLAIN ANALYZE SELECT ar.name AS artist_name, SUM(il.unit_price * il.quantity) AS
total_revenue FROM invoice_line il JOIN track t ON il.track_id = t.track_id JOIN album al ON
t.album_id = al.album_id JOIN artist ar ON al.artist_id = ar.artist_id GROUP BY ar.name ORDER
BY total_revenue DESC;
```

```
CREATE INDEX IF NOT EXISTS idx_invoice_line_track_id ON invoice_line(track_id); CREATE
INDEX IF NOT EXISTS idx_track_album_id ON track(album_id); CREATE INDEX IF NOT EXISTS
idx_album_artist_id ON album(artist_id);
```

Performance comparison:

Finding: adding indexes on the join columns made no meaningful difference to query execution time, and the query planner continued to use sequential scans on every table rather than the new indexes. This is expected behavior at this scale — the largest table involved (track) has only 3,503 rows, well below the size at which an index lookup becomes cheaper than simply scanning and hashing the whole table in memory. The main takeaway for query optimization: indexing is not a universal performance fix; the PostgreSQL query planner already makes a cost-based decision, and on small tables it correctly prefers sequential scans over index scans. Indexing benefits become more apparent on tables with significantly larger row counts.

7. Summary of Key Insights

Revenue concentration differs sharply between datasets: sales_data shows a small number of dominant accounts (Euro Shopping Channel alone accounts for \$912,294.11, driven by order frequency rather than order size), while Chinook customer spend is fairly evenly distributed across all 59 customers.

Referential integrity in Chinook is clean: INNER, LEFT, and RIGHT joins between customer and invoice all return identical results, with no missing or orphaned records on either side.

Roughly 37% of Chinook customers (22 of 59) spend above the database average, identified using a nested subquery that first aggregates per-customer totals before averaging them.

ROW_NUMBER and RANK behave identically except when ties occur — RANK assigns shared ranks to tied values and skips subsequent numbers (confirmed directly: a 29-way tie at \$37.62 all received rank 29, with the next customer jumping to rank 59), while ROW_NUMBER always assigns a unique, sequential value regardless of ties.

PARTITION BY enables per-group ranking (track length within each genre) without collapsing rows the way GROUP BY does; the rank correctly reset to 1 at every genre boundary across all 25 genres in the catalog, with Rock being by far the largest genre at 1,297 tracks.

Chinook's data is synthetic and evenly distributed rather than reflecting real-world variation: monthly revenue stays flat at almost exactly \$37.62 across five years, and 58 of 59 customers made exactly 7 purchases each — a pattern unlikely to occur in genuine transactional data.

sales_data, by contrast, shows authentic retail seasonality: revenue spikes sharply every November (over \$1,000,000 in both 2003 and 2004) with October also elevated, consistent with pre-holiday buying in a toy and collectibles business, alongside a general year-over-year upward trend.

Classic Cars is the standout product line in sales_data, generating \$3,919,615.66 — more than double the next-highest category — while Iron Maiden leads Chinook's artist revenue at \$138.60, with two TV shows (Lost, The Office) notably appearing among the top 10 grossing “artists.”

Indexing did not improve performance on these dataset sizes; the query planner already selects the most efficient scan strategy, and indexing pays off mainly as table size grows significantly beyond what is used here.